John Cheek
Alexander Card
CSC 481
21 October 2025

Milestone 3 Writeup

Deciding on an object model was more difficult than I expected it to be. Before ultimately deciding on the entity-component model our team thoroughly considered all the possible object models. We decided against the object centric and property centric models relatively quickly. The object centric model is pretty similar to what we had already and we already saw some of its limitations when implementing the networking functionality in milestone 2. Mainly, it was difficult to ensure consistent object behavior across all clients, and any future object types we wanted to implement would present similar challenges. We discussed using a property centric model as we really liked the flexibility it provided, but we couldn't come up with a design where our current object model and game object behaviors translated into the property centric model. While feasible, implementing a property centric model would require a very large scale refactor in order to preserve existing engine functionality, so our team felt another model would be better in order to make sure our engine's features are preserved. Ultimately, we leaned towards a component centric model as we decided relegating most engine functionality to object components could lead to easier implementation of future engine functionality with easy to add components used to describe specific object behaviors, while putting less pressure on client-server communication.

Of the multiple types of component models we decided mostly on a pure component model with some features of a generic component model, leading to a hybrid 'entity-component system.' With the pure-component model we could instantiate entities using our original 'Entity' class as our factory class and give each object components that would describe its behaviors and interactions with other game objects. Furthermore, we could move some of our existing engine systems onto components to facilitate creation of new game objects and easily attribute specific behaviors to them. For instance, giving the 'RigidBody' component to a game object you can enable physics on it and by giving the 'TextureRenderer' component (along with a pointer to the renderer, a path to a specific media, and information about the sprite sheet) you can easily give an component an animated sprite. This was the main advantage of using a pure-component model. However, instead of having a distinct UUID like a traditional pure-component model, the entity's name string functions as its unique identifier. While we acknowledge that string comparisons are expensive, our previous object centric model (and thus all of our previous engine functionality) used them and they make future development and identifying specific entities more intuitive. Ultimately, our new game object model landed on an Entity-Component System. In this system, all game objects are considered entities (though the terms can be used interchangeably) and each entity can support any component.

Currently, our entity-component game object system supports the following components: BoxCollider, DeathZone, DebugRenderer, Health, RigidBody, ScreenAnchor, SideScroll, SpawnPoint, TextureRenderer, and TouchDamage. BoxCollider gives an object an axis-aligned bounding box collision (Figure 1). This component handles collision detection and resolution. The component uses the Entity's assigned width and height (designated in the factory class) but supports a custom collision box size if desired. This component is given to any entity that needs to be 'solid' (a player cannot pass through it and must stop movement when a collision is

detected). DeathZone describes an entity where, when the player collides with it, they are teleported to either a specified spawn point or the closest spawn point. This component is used for hazards, pits, and respawn logic. DebugRenderer is primarily used for debug purposes and draws a colored square for an entity that allows visualizing object bounds for testing during development. Health tracks the health value of an entity. It supports taking damage, healing, and death logic. This is currently only used for the player character but can also be used for enemy logic. RigidBody is used to enable physics on an entity. Currently, the engine's only physics feature is gravity, but other physics systems will also be enabled by this component. This component can also change the strength gravity has on a specific entity. ScreenAnchor and SideScroll are both used for the side scrolling logic. ScreenAnchor anchors an entity to a specific position (useful for UI elements like the pause button), and SideScroll describes the vertical side-scrolling boundaries on each side of the screen which will shift the camera when collided with. SpawnPoint designates a player respawn location. It is used by the DeathZone to determine where the player should reappear if killed. TextureRenderer assigns a sprite to an entity. Developers can load a static texture by giving a pointer to the SDL Renderer and the path to the sprite, and load an animated sprite by passing additional arguments (like the frame count in the sprite sheet, the size of each frame, and how fast the animation should play). Figure 3 shows an entity being instantiated with an animated texture using this component. Lastly, TouchDamage is a component that assigns the specified damage value to entities that collide with this component. This component can be used for spikes, enemies, and other environmental hazards.

Another addition of our component model that is not usually present in a pure-component model is the use of 'Tags.' Tags are used to describe an entity's generic behavior. In our previous object centric model instead, we used subclasses to describe a specific "type" of entity. We can use entity "tags" for a similar purpose. For instance, a static platform, a generic entity that has the BoxCollider component which stays stationary and the player can stand on and collide has the "Ground" tag. All other platforms that have this behavior also have the "Ground" tag. While usually tags are just for object classification, some tags can be used by components to describe its behavior with other components. As an example, the DeathZone component checks for either an object with a specific Spawn tag (e.g. "SpawnA"), or the closest object with the "Spawn" tag to allow the player to respawn at a spawn point. (see Figure 2 for the Death Zone component using the Spawn tag for its behavior).

Scalability and future development are one of the biggest strengths of a component based object model. In order to add new game components, we can simply create a new component and then add those components to other objects we want to share that new behavior. Furthermore, adding a new component does not really require modifying existing code (except in a few cases) the developer just needs to create a new component and add that component to an entity. This behavior is also kept off the network, as the components themselves are used to describe the entity's behavior and interactions. The client just needs to instantiate the entity and add its components. One of the places where the scalability of this model shined was in adding the individual game functionality for part 2 of the milestone. To create the spawn points, death zones, and side scrolling boundaries, we just had to create their respective components and then attach them to a generic invisible entity (See Figure 1 and Figure 2 for the implementation).

Our modified entity component model is not without limitations though. The main one we saw is that direct communication between components attached to the same entity is limited; components typically operate independently and rely on other engine systems to determine interactions. For example, the Health component cannot directly notify a TextureRenderer to

change the sprite when damage is taken, instead it must rely on other engine systems to coordinate that change. While this separation makes the engine more modular and flexible, it can make implementing complex component interaction behaviors more difficult and indirect. However, a property-centric object model would face the same limitation as properties do not directly communicate, and any cross-property behavior would still require additional logic or other systems to manage property interactions, resulting in similar challenges for complex game mechanics.

We decided to use a client-server model for our game engine. As our team members are all in CSC 481, this was the only networking model we developed. Initially, we were concerned about the process of refactoring the network model to use the new game object system but the strengths of our entity-component system made the refactor rather simple. We could reuse the old entity class as the factory for all objects (which our network model was already doing) and then assign the necessary components to each object. Figure 4 shows the client creating local entities based on the data held on the server and attaching components to them. While the component tags put the majority of entity behavior off the network and onto the entity-component system, the clients and server do of course need to communicate with each other. It does this by sending formatted commands using a request-reply pattern. The client collects local input and game state changes then packages them into a command string and sends it to the server. The current commands are CONNECT|(playerName), DISCONNECT|(playerName), UPDATE|(entity name)|(new entity x)|(new entity y), and PAUSE. Connect tells the server a new client has connected, disconnect tells the server when a client has disconnected and gracefully handles the disconnection by removing the disconnected player entity from the server and all clients (which funnily enough was something I implemented in order to fix a bug in Milestone 2, not knowing it would be a requirement for this one), update sends the server an entity's new position, and pause tells the server to broadcast the paused state to all clients. The server is constantly processing requests from all the clients and sending a reply when the command is completed. Figure 5 shows the server processing the update, pause, and disconnect commands from its clients.

In the end, when developing my individual game using the new entity-component system, I was pleasantly surprised by how intuitive the new ECS was to use and just how much it facilitated the creation of my individual game. Instantiating entities is pretty similar to how it has been in the past two milestones, but the addition of components meant that no additional code needed to be changed to support the game’s behaviors. Perhaps the most significant challenge was simply implementing the new game object model. With such a large scale refactor, a lot of work had to be done in order to preserve the existing engine functionality. Thankfully, our engine design already had core systems like physics and collisions in place outside of the entity classes, so we could just tailor the new components to use those existing engine systems– ultimately redistributing the code into an accessible place where it can be modified and easily reused by all entities with those components. Another thing our team didn’t really consider in designing is the components that rely on other components. For instance, the rigid body component did not account for moving platforms carrying the player, the side scrolling boundary components needed to use other components to function properly, and the death zone components needed to rely on specific spawn point components. To overcome these challenges, more specific code had to be added to those components. While this is not ideal as our team wanted to keep the component system as simple and scalable as possible, it did teach us that there is potential for more complex components that work together with other components and tags to ultimately create more advanced game engine features. However; we are yet to see the limitations of what

exactly our components can and can’t do. Another significant challenge will be porting the additional game components to function over networking as we only developed the extra game components for a single player game. While the death zone and spawn point components should function smoothly, I can foresee challenges with using the side scrolling components in a multiplayer game and synchronizing the side scrolling behavior across multiple clients. We faced a similar challenge with the pause functionality and to overcome that we added a new command type, so that will likely be necessary for the sidescrolling boundaries as well. Nevertheless, I am very proud of our team's entity-component system design and how well it facilitates the creation of distinct game components. To create my individual game project for milestone 3 ‘Avoid the Void’ I simply needed to create new entities and give them the necessary components and tags. It was very rewarding to have all the hard work on the new game object model pay off when they facilitated the creation of a 2D platformer level. Figure 6 shows a gameplay screenshot of the game with debug components enabled in order to visualize the side scrolling, spawn point, and death zone components.

```cpp
SDL_FRect aabb() const {
    const Entity* e = getOwner();
    SDL_FRect r{
        e->getX() + offsetX_,
        e->getY() + offsetY_,
        static_cast<float>(useCustom_ ? customW_ : e->getWidth()),
        static_cast<float>(useCustom_ ? customH_ : e->getHeight())
    };
    return r;
}

// Resolve overlap against another collider (using minimal push-out).
// If vertical resolution occurs, zero out vertical velocity on the owne
bool resolveAgainst(const BoxCollider& other) {
    Entity* a = getOwner();
    Entity* b = other.getOwner();
    if (!a || !b) return false;

    SDL_FRect A = aabb();
    SDL_FRect B = other.aabb();

    // No overlap
    const bool sep =
        (A.x + A.w) <= B.x ||
        (B.x + B.w) <= A.x ||
        (A.y + A.h) <= B.y ||
        (B.y + B.h) <= A.y;
    if (sep) return false;

    // Compute minimal translation vector
    float axCenter = A.x + A.w * 0.5f;
    float ayCenter = A.y + A.h * 0.5f;
    float bxCenter = B.x + B.w * 0.5f;
    float byCenter = B.y + B.h * 0.5f;

    float dx = axCenter - bxCenter;
    float dy = ayCenter - byCenter;

    float px = (A.w * 0.5f + B.w * 0.5f) - std::fabs(dx);
    float py = (A.h * 0.5f + B.h * 0.5f) - std::fabs(dy);

    if (px < py) {
        // Resolve on X
        float sx = (dx < 0.f) ? -1.f : 1.f;
        a->x += sx * px;
    }
    else {
        // Resolve on Y
        float sy = (dy < 0.f) ? -1.f : 1.f;
        a->y += sy * py;
        a->velY = 0.f; // snap ground/ceiling stops vertical motion
    }
    return true;
```

Figure 1: Axis-aligned bounding box collision detection, used by the BoxCollider component

```cpp
class DeathZone : public ecs::Component {

public:
    // If spawnName is empty, the nearest entity with tag "SPAWN" will be used.
    DeathZone(const std::string& spawnName = "") : spawnName_(spawnName) {}

    void onUpdate(float dt) override {
        Entity* player = EntityManager::getInstance().findEntityByName("Player");
        if (!player) return;

        Entity* owner = getOwner();
        if (!owner) return;

        if (collisionDetection(*owner, *player)) {
            Entity* spawn = nullptr;

            // Try to find spawn by name
            if (!spawnName_.empty()) {
                spawn = EntityManager::getInstance().findEntityByName(spawnName_);
            }

            // If not found by name, find the nearest spawn with "SPAWN" tag
            if (!spawn) {

                // Find the nearest spawn point
                float nearestSpawn = std::numeric_limits<float>::max();

                for (auto* entities : EntityManager::getInstance().entities) {
                    if (!entities) continue;

                    // Check for "SPAWN" tag
                    if (entities->hasTag("SPAWN")) {
                        float dx = (entities->x + entities->width*0.5f) - (player->x + player->width*0.5f);
                        float dy = (entities->y + entities->height*0.5f) - (player->y + player->height*0.5f);
                        float d = std::sqrt(dx*dx + dy*dy);
                        if (d < nearestSpawn) {
                            nearestSpawn = d;
                            spawn = entities;
                        }
                    }
                }
            }

            // Teleport player to spawn point
            if (spawn) {
                player->x = spawn->x;
                player->y = spawn->y;
                player->velY = 0.0f;
                Camera::getInstance().smoothTo(spawn->x - 200.0f, spawn->y - 800.0f, 0.4f);
                SDL_Log("DeathZone: moved player to spawn '%s' at (%.1f, %.1f)", spawn->name.c_str(), spawn->x, spawn->y);
            }

            else {
                SDL_Log("DeathZone: no spawn found to teleport player");
```

Figure 2:The DeathZone component using entities with the “Spawn” tag

```cpp
auto* theVoid = new Entity("TheVoid", 5000.0f, 200.0f, 576, 576);
theVoid->setTag("DEATH");
theVoid->addComponent<DeathZone>("SpawnC");
theVoid->addComponent<TextureRenderer>(renderer, "media/darkworld_spawn_swirlingorb.png", 128, 128, 4, 1, 6.6667f);
manager.addEntity(theVoid);
```

Figure 3: Instantiating a DeathZone entity with a specific spawn location

```
Entity* e = manager.findEntityByName(name);
if (e) {
    // Always update position/size from server
    e->x = x;
    e->y = y;
    e->width = w;
    e->height = h;
}
else {
    // Instantiate components from server based on reported type
    if (type == "PLAYER") {
        auto* e = new Player(name, x, y, w, h);
        e->setTag("PLAYER");
        e->addComponent<TextureRenderer>(renderer, "media/darkworld_enemy_nyx_idle.png", 128, 128, 8, 1, 6.6667f);
        e->addComponent<BoxCollider>();
        manager.addEntity(e);
    }
    else if (type == "GROUND") {
        auto* e = new Entity(name, x, y, w, h, false, true);
        e->setTag("GROUND");
        e->addComponent<TextureRenderer>(renderer, "media/darkworld_platform_mossystone.png");
        e->addComponent<BoxCollider>();
        manager.addEntity(e);
    }
    else if (type == "MOVING_PLATFORM") {
        auto* e = new MovingPlatform(name, x, y, w, h, true, 150.0f, 400.0f);
        e->setTag("MOVING_PLATFORM");
        e->addComponent<TextureRenderer>(renderer, "media/darkworld_platform_mossystone.png");
        e->addComponent<BoxCollider>();
        manager.addEntity(e);
    }
    else if (type == "PAUSEBUTTON") {
        auto* e = new PauseButton(name, x, y, w, h, &timeline);
        e->setTag("PAUSEBUTTON");
        e->addComponent<TextureRenderer>(
            renderer, "media/darkworld_enemy_skullduggery_idle.png",
            32, 32, 6, 1, 6.6667f
        );
        manager.addEntity(e);
    }
    else {
        auto* e = new Entity(name, x, y, w, h, false, false, type);
        e->setTag(type);
        manager.addEntity(e);
    }
}
```

Figure 4: The client retrieving entities from the anxiety manager and instantiating them client side with the necessary components

```
if (cmd == "UPDATE" && !name.empty()) {
    try {
        float x = std::stof(xs), y = std::stof(ys);
        std::lock_guard<std::mutex> lock(entityMutex);
        Entity* e = EntityManager::getInstance().findEntityByName(name);
        if (e) {
            e->x = x;
            e->y = y;
        }
        else {
            auto* player = new Entity(name, x, y, 128, 128, true, false, "PLAYER");
            player->setTag("PLAYER");
            EntityManager::getInstance().addEntity(player);
        }
        reply_str = "OK";
    }
    catch (...) { reply_str = "ERR"; }
}
else if (cmd == "PAUSE") {
    // Toggle authoritative server timeline pause state
    gameTimeline.togglePause();
    bool paused = gameTimeline.isPaused();
    std::cout << "[Server] PAUSE command received from " << name << ". Now paused = " << (paused ? "true" : "false") << std::endl;
    reply_str = paused ? "PAUSED" : "RUNNING";
}
else if (cmd == "DISCONNECT" && !name.empty()) {
    {
        std::lock_guard<std::mutex> lock(entityMutex);
        Entity* e = EntityManager::getInstance().findEntityByName(name);
        if (e) {
            EntityManager::getInstance().removeEntity(e);
            std::cout << "[Server] Client disconnected: Player Name=\"" << name << "\". Entity removed." << std::endl;
        }
        else {
            std::cout << "[Server] Client disconnected: Player Name=\"" << name << "\". No entity found to remove." << std::endl;
        }
    }
    reply_str = "BYE";
    zmq::message_t reply(reply_str.size());
    memcpy(reply.data(), reply_str.data(), reply_str.size());
    socket.send(reply, zmq::send_flags::none);

    break;
}
zmq::message_t reply(reply_str.size());
memcpy(reply.data(), reply_str.data(), reply_str.size());
if (!socket.send(reply, zmq::send_flags::none)) {
    // If send fails, treat as disconnect
    break;
}
```

Figure 5: The server logic for processing the Update, Pause, and Disconnect commands

Figure 6: A gameplay screenshot of 'Avoid the Void' with debug highlights enabled to show invisible components